BIOINFORMATICS

Workshop 4: Viterbi, Forward and Backward

Companion to Lecture 4: Decoding and Evaluation: Viterbi and Forward-Backward

Prof. Magnus Bordewich

Learning Objectives

- Implement the Viterbi algorithm to decode the most probable hidden state path.
- Implement the forward and backward algorithms.
- Combine forward and backward to compute posterior state probabilities.

Background

All three algorithms share the same recursive skeleton over the HMM class from Workshop 3: fill a table indexed by position and state, one position at a time. Viterbi takes a max where forward takes a sum; backward runs the recursion from the end of the sequence backwards.

Coding Activity

1. Add a method .viterbi(symbols) to your HMM class returning (best_path, best_prob): fill v_l(i) = e_l(x_i) * max_k[v_k(i-1) * m_kl], remember the argmax at each step, and trace back. Work in log-space to avoid underflow on longer sequences.
2. Reproduce the lecture's worked example directly: build the two-state fair/biased coin HMM with m_FF=0.75, m_FB=0.25, m_BF=0.25, m_BB=0.75, e_F(H)=0.5, e_F(T)=0.5, e_B(H)=0.75, e_B(T)=0.25 and equal start probabilities, run .viterbi() on x = "TTHHT" and “TTHHT”, and confirm you recover the same decoded path and probability as the lecture slide.
3. Test .viterbi() on a sequence simulated from your Workshop 3 dishonest casino of CpG HMM. Compare the recovered state path to the true, simulated state path.
4. Add .forward(symbols) returning the full table F_l(i) and P(x) = sum_k F_k(L), and .backward(symbols) returning B_l(i), initialised with B_l(L) = 1 for every state - both in log-space.
5. Add .posterior(symbols) combining forward and backward to return P(Si=k | x) for every position and state, and check these sum to 1 across states at every position. Confirm P(x) and the posteriors match the lecture's worked table for x = "TTHHT".

Extension Tasks (Optional)

a) Write count_transitions/count_emissions and mle_estimate(M_counts, E_counts, pseudocount=0) implementing counting with optional pseudocount smoothing.

b) Add a method .expected_counts(symbols) to your HMM class that runs .forward()/.backward() and returns <M_kl>, <E_k(b)> using the lecture's formulas.

c) Write baum_welch(hmm, sequences, iterations, pseudocount=0) that repeats: sum expected counts across all training sequences, then re-estimate via mle_estimate, for the given number of iterations.

d) Simulate 30 sequences of length 1000 from your Workshop 3 CpG HMM, keeping only the emitted symbols (discard the true state paths). Initialise a fresh HMM with deliberately 'wrong' starting parameters, run baum_welch for 15-20 iterations, and compare the learned parameters to the true ones. Track the total log-likelihood each iteration and confirm it never decreases.

e) Run Baum-Welch from three different random initialisations on the same training data and compare the final learned parameters - do all three converge to (approximately) the same model?

Looking Ahead

Next we change topic and look at protein folding.